Analiza obsługi klienta — Website Expert

> Data: 2026-04-19 Zakres: jak Website Expert pozyskuje, modeluje, obsługuje i rozwija relację z klientem w CRM, portalu i self-service SaaS.

---

1. Podsumowanie wykonawcze

Obsługa klienta w Website Expert jest już szeroka, ale nie jest jednolita domenowo. System używa jednego rekordu `Client` jako wspólnego punktu dla prospecta, aktywnego klienta agencyjnego i klienta z dostępem do portalu, a osobno używa `User` jako tożsamości logowania. To działa dobrze dla MVP, bo szybko spina lead capture, CRM, wyceny, kontrakty, projekty, płatności i portal. Jednocześnie miesza dwa różne modele biznesowe:

1. klient agencyjny obsługiwany przez CRM,
2. klient self-service korzystający z workspace, billing i growth tools.

Najważniejszy wniosek: projekt już umie prowadzić klienta od pierwszego kontaktu do podpisania umowy i realizacji projektu, ale ma kilka szczelin na styku `CRM -> portal -> business workspace`, które trzeba domknąć przed dalszym skalowaniem.

---

2. Jak system rozumie klienta

2.1 Główne encje

Encja	Rola w systemie	Fakty z kodu
`Lead`	szansa sprzedażowa / inbound enquiry	`app/Models/Lead.php`, `app/Services/Leads/LeadService.php`
`Client`	rekord CRM dla firmy lub osoby kontaktowej	`app/Models/Client.php`, `app/Filament/Resources/ClientResource.php`
`Contact`	dodatkowa osoba po stronie klienta	tworzona przez `CreateLeadAction`, ale nie ma osobnego UI listy kontaktów
`User`	konto logowania do portalu / self-service	`app/Models/User.php`
`Business`	workspace SaaS / tenant	`app/Services/Business/BusinessService.php`, `app/Helpers/BusinessHelper.php`

2.2 Kluczowy model domenowy

Aktualny model nie robi twardego rozdziału między prospectem a klientem. Zamiast tego:

- pierwszy inbound kontakt tworzy lub znajduje rekord `Client`,
- ten rekord startuje ze statusem `prospect`,
- wygrany lead promuje klienta do `active`,

- portal działa przez powiązanie `clients.portal_user_id -> users.id`.

To oznacza, że tabela `clients` pełni dziś rolę zarówno:

- prospect registry,
- customer master record,
- punktu wejścia do portalu.

2.3 Co jest osią relacji

Najważniejsze pole w praktyce to `primary_contact_email`.

- `CreateLeadAction` szuka klienta po `primary_contact_email` oraz opcjonalnie `business_id`.
- Jeśli klient nie istnieje, tworzy go automatycznie jako `prospect`.
- Portal logowania nie działa przez osobny `ClientPortalAccount`, tylko przez zwykłego `User`, którego system przypina do klienta przez `portal_user_id`.

To upraszcza MVP, ale słabiej skaluje się do wielu kontaktów po stronie jednego klienta.

---

3. Lifecycle klienta w obecnym kodzie

3.1 Pozyskanie klienta / prospecta

Projekt ma kilka punktów wejścia:

Kanał wejścia	Jak działa	Kluczowe pliki
Public landing page	deduplikacja 24h, UTM, consent, source attribution, increment `conversions_count`	`app/Http/Controllers/Leads/LeadCaptureController.php`, `app/Services/Leads/PublicLeadCaptureService.php`, `app/Services/Leads/LeadService.php`
Kontakt publiczny	zwykły formularz kontaktowy tworzy lead i consent	`app/Http/Controllers/ContactController.php`
Quick CTA z usług	skrócony formularz z service pages	`app/Http/Controllers/ContactController.php`
Kalkulator	lead z `calculator_data`, estimate i source=`calculator`	`app/Http/Controllers/CalculatorLeadController.php`
API	leady z integracji zewnętrznych pod token biznesu	`app/Http/Controllers/Api/LeadCaptureController.php`

Kanał wejścia	Jak działa	Kluczowe pliki
Rejestracja / social login	tworzy `User`, `Client`, rolę `client` i `Business`	`app/Http/Controllers/Auth/RegisteredUserController.php`, `app/Http/Controllers/Auth/SocialAuthController.php`

Co dzieje się technicznie po capture

`CreateLeadAction` wykonuje trzy ważne kroki:

1. znajduje lub tworzy `Client`,
2. znajduje lub tworzy `Contact`,
3. tworzy `Lead` w pierwszym etapie pipeline.

To znaczy, że system bardzo wcześnie materializuje prospecta jako `Client`. Zaletą jest spójność relacji. Wadą jest to, że CRM klienta szybko puchnie rekordami, które nigdy nie zostaną prawdziwymi klientami.

3.2 Prowadzenie klienta w CRM

Po utworzeniu leada system obsługuje klienta przez moduł CRM:

- przypisanie właściciela leada,
- zmianę stage,
- oznaczenie lead jako `won` lub `lost`,
- aktywność i timeline,
- automatyzacje zależne od triggerów.

Kluczowe fakty:

- `LeadService::assign()` loguje aktywność i emituje `LeadAssigned`.
- `LeadService::updateStage()` loguje zmianę etapu.
- `LeadService::markWon()` ustawia `won_at`, przenosi do etapu `is_won=true` i promuje klienta z `prospect` do `active`.
- `LeadService::markLost()` ustawia `lost_at` i przenosi do etapu `is_lost=true`.

W praktyce to właśnie `markWon()` jest momentem, w którym prospect staje się klientem w sensie operacyjnym.

3.3 Komunikacja i automatyzacje

Obsługa klienta nie kończy się na CRM. Projekt ma pełny silnik automatyzacji oparty o eventy i kolejkę.

Triggery używane w relacji z klientem

- `lead.created`
- `lead.assigned`
- `lead.stage_changed`
- `lead.won`
- `lead.lost`

- `project.created`
- `project.status_changed`
- `invoice.sent`
- `invoice.paid`
- `quote.sent`
- `quote.accepted`
- `contract.created`
- `contract.sent`
- `contract.signed`

Źródło: `app/Listeners/AutomationEventListener.php`.

Preferencje komunikacji klienta

Klient ma cztery główne przełączniki komunikacyjne:

- `notify_email_transactional`
- `notify_email_projects`
- `notify_email_marketing`
- `notify_sms`

Gatekeeping robi `ClientNotificationGate`, a klient sam może edytować te ustawienia w portalu przez `Portal/NotificationController`.

Kanały kontaktu już obecne

- e-mail systemowy,
- e-mail marketing/automation,
- SMS przez Twilio,
- powiadomienia adminom przy akcjach klienta,
- timeline aktywności klienta.

3.4 Sprzedaż i formalizacja współpracy

Website Expert prowadzi klienta przez kilka etapów domknięcia sprzedaży:

Sales offers

- klient dostaje publiczny link token-based do oferty,
- otwarcie oferty oznacza `viewed`,
- kliknięcie CTA zapisuje akceptację i uruchamia notyfikacje do adminów,
- klient dostaje mail potwierdzający.

Pliki: `app/Http/Controllers/ClientSalesOfferController.php`, `app/Services/SalesOfferService.php`.

Quotes

- klient w portalu może zaakceptować lub odrzucić wycenę,
- status zmienia się bezpośrednio z poziomu portalu,

- wydarzenie wpada do aktywności i automatyzacji.

Plik: `app/Http/Controllers/Portal/QuoteController.php`.

Contracts

- klient widzi umowę w portalu,
- może ją podpisać elektronicznie,
- system zapisuje `signed_at`, `signer_name`, `signer_ip`, opcjonalnie `signature_data`.

Plik: `app/Http/Controllers/Portal/ContractController.php`.

Invoices i płatności

- klient widzi tylko niedraftowe faktury,
- może wejść w wybór metody płatności,
- obsługiwane są Stripe i PayU,
- wynik płatności wraca do portalu.

Pliki: `app/Http/Controllers/Portal/InvoiceController.php`, `app/Http/Controllers/Portal/PaymentController.php`, `app/Services/PayuService.php`.

3.5 Portal klienta

Portal klienta działa pod `/portal` i jest oparty o `BasePortalController`, który odnajduje klienta po `portal_user_id`.

Funkcje portalu potwierdzone w kodzie:

- dashboard z projektami, fakturami, wycenami i timeline (`Portal/DashboardController`),
- projekty i messaging (`Portal/ProjectController`),
- wyceny (`Portal/QuoteController`),
- umowy (`Portal/ContractController`),
- faktury i płatności (`Portal/InvoiceController`, `Portal/PaymentController`),
- preferencje komunikacji (`Portal/NotificationController`),
- szczegóły leadów wygenerowanych z landing pages (`Portal/LeadController`),
- billing i plan SaaS (`Portal/BillingController`),
- growth tools: landing pages, AI generator, business profile, API tokens (`resources/js/Layouts/PortalLayout.jsx`).

To jest ważne: portal nie jest wyłącznie „klasycznym client portalem”. To także wejście do workspace produktu.

3.6 Self-service SaaS

Rejestracja i social login tworzą jednocześnie:

- `User` z rolą `client`,
- rekord `Client`,
- własny `Business`,
- membership w `business_users` z rolą `owner`.

To powoduje, że klient self-service od razu ma pełny workspace i może używać:

- landing pages,
- AI generatora,
- ustawień firmy,
- billing i planów,
- tokenów API.

Ta ścieżka jest znacznie pełniejsza niż ścieżka klienta agencyjnego zapraszanego później do portalu.

3.7 Token-based klient-facing flows poza portalem

Poza logowaniem istnieją dwa niezależne kanały obsługi klienta bez auth:

- `client/briefings/{token}` — klient wypełnia briefing, autosave + submit,
- `offers/{token}` — klient ogląda ofertę i akceptuje CTA.

To daje wygodny low-friction entry, ale rozbija doświadczenie klienta na osobne wejścia: portal i publiczne linki tokenowe.

---

4. Co jest już mocne

4.1 End-to-end lifecycle istnieje naprawdę

Projekt nie kończy się na lead capture. Ma realny ciąg:

`lead -> CRM -> quote/offer -> contract -> project -> invoice/payment -> portal`

To jest bardzo dobra baza pod produkt usługowy i agencyjny SaaS.

4.2 Dane klienta są spinane wcześnie

Automatyczne tworzenie `Client` przy capture powoduje, że od pierwszego kontaktu system ma jeden rekord odniesienia dla leadów, kontaktów, projektów, faktur i umów.

4.3 Komunikacja jest już segmentowana

Rozdzielenie transactional / project / marketing / SMS to dobry fundament pod compliance i customer experience.

4.4 Portal robi coś więcej niż podgląd PDF-ów

Klient może:

- pisać wiadomości w projekcie,
- akceptować wyceny,
- podpisywać kontrakty,
- płacić faktury,
- zmieniać preferencje komunikacyjne,
- w modelu self-service także generować landing pages i zarządzać workspace.

4.5 Jest activity trail klienta

`ClientActivityListener` buduje osobny timeline klienta na podstawie leadów, projektów, faktur, wycen i kontraktów. To jest dobra baza pod customer success i account management.

---

5. Najważniejsze luki i ryzyka

5.1 Mieszanie dwóch typów klienta w jednym modelu

System łączy w jednym UX i jednej encji:

- klienta agencyjnego,
- prospecta z formularza,
- klienta self-service z własnym workspace.

Na dziś to działa, ale zaciera granice odpowiedzialności i utrudnia precyzyjne policy, onboarding i feature gating.

5.2 Automatyczny invite do portalu ma zły URL

`CreatePortalAccessAction` wysyła login URL z końcówką `/client`, ale w trasach nie ma takiego endpointu. Portal działa pod `/portal`, a standardowe logowanie pod `/login`.

To oznacza, że automatyczne zaproszenie do portalu jest dziś logicznie niespójne.

Pliki:

- `app/Automation/Actions/CreatePortalAccessAction.php`
- `routes/web.php`

5.3 Zapraszany klient agencyjny nie dostaje membership w `Business`

Self-service registration tworzy `BusinessUser`, ale `CreatePortalAccessAction` i `CreatePortalUser` tego nie robią.

Skutek:

- klient agencyjny może mieć konto portalu,
- ale nie musi mieć `currentBusiness()`,
- a część funkcji portalu opiera się właśnie na `currentBusiness()`.

Dotyczy to szczególnie:

- billing,
- growth tools,
- landing pages,
- szczegółu leadów w portalu.

To jest dziś najważniejsza luka architektoniczna na styku agency client vs SaaS workspace.

5.4 Portal pokazuje sekcje biznesowe wszystkim zalogowanym użytkownikom

`PortalLayout.jsx` renderuje growth tools i business settings bez rozróżnienia, czy użytkownik:

- jest self-service ownerem biznesu,
- jest zaproszonym klientem agencyjnym,
- ma aktywne membership w `business_users`.

To może prowadzić do pustych ekranów, redirect loops lub UX, które wygląda na zepsute.

5.5 Autoryzacja leada w portalu jest oparta o business, nie o klienta

`Portal/LeadController` sprawdza `lead.business_id === currentBusiness()->id`, ale nie potwierdza `lead.client_id === current client`.

W modelu jednoosobowego self-service to jest akceptowalne. W modelu wielu użytkowników w jednym biznesie lub przy przyszłym multi-contact portalu to jest potencjalny wyciek danych między klientami.

5.6 `ClientResource` nie jest centrum pracy z klientem

Obecny `ClientResource` jest raczej kartą danych niż hubem relacji.

Potwierdzone ograniczenia:

- brak relation managers w `app/Filament/Resources/ClientResource/`,
- `getRelations()` zwraca pustą tablicę,
- brak szybkiego spójnego widoku leadów, projektów, wycen, faktur i kontaktów z poziomu klienta,
- `getEloquentQuery()` nie dokłada scopa po `business_id`.

To utrudnia account management i pracę opiekuna klienta.

5.7 Jeden portal user na klienta to za mało

Relacja `portal_user_id` narzuca jeden login na jednego klienta. Model `Contact` istnieje, ale nie służy do wielu dostępów portalowych.

Brakuje:

- multi-user access dla klienta,
- ról po stronie klienta,
- zapraszania kilku kontaktów,
- ograniczeń typu billing-only / contracts-only / marketing-only.

5.8 Nie wszystkie inbound leady są jednoznacznie tenant-scoped

`landing_page` i `api` są dobrze osadzone w biznesie. Publiczny kontakt i kalkulator przekazują `currentBusiness()`, więc w scenariuszach publicznych często kończą z `business_id = null`.

Dla single-business deployment to jest do przeżycia. Dla skalowanego SaaS to słaba izolacja i trudniejsza analityka.

5.9 Doświadczenie klienta jest rozproszone

Klient może wchodzić przez:

- portal po loginie,
- publiczny link do briefingu,
- publiczny link do sales offer,
- maile z wyceną / kontraktem / fakturą.

Każdy kanał ma sens osobno, ale dziś nie buduje jednego, spójnego customer workspace.

---

6. Co warto dodać i usprawnić

6.1 Quick wins

1. Naprawić invite flow do portalu

- ujednolicić URL logowania w invite mailach,
- dodać jeden serwis `PortalAccessService`,
- obsłużyć resend invite, reset hasła i status aktywacji.

2. Rozdzielić tryb klienta agencyjnego i self-service

Minimum na teraz:

- feature-gating w `PortalLayout`,
- ukrywanie growth tools i billing dla userów bez `BusinessUser`,
- osobny empty state dla klienta portalowego bez workspace.

3. Dociągnąć membership biznesowe przy invite

Jeśli klient agencyjny ma widzieć leady, billing lub workspace-level funkcje, to konto musi być świadomie dołączane do `business_users`. Jeśli nie ma ich widzieć, interfejs też musi to odzwierciedlać.

4. Zaostrzyć autoryzację portalu

- policy dla `PortalLeadController`,
- jawne sprawdzenie `client_id`,
- middleware lub dedicated guard dla client portal routes.

5. Zrobić z `ClientResource` prawdziwe centrum relacji

Dołożyć relation managers lub custom tabs dla:

- kontaktów,
- leadów,
- projektów,
- wycen,
- faktur,
- kontraktów,

- aktywności klienta.

6.2 Usprawnienia produktowe średniego horyzontu

6. Wielu użytkowników po stronie klienta

Warto wprowadzić:

- `ClientPortalMember` albo wykorzystanie `Contact` jako konta portalu,
- role typu `owner`, `billing`, `legal`, `marketing`,
- osobne invite i revoke access.

7. Jednolity customer timeline

Scalić w jednym widoku:

- leady,
- oferty,
- wyceny,
- kontrakty,
- wiadomości,
- płatności,
- briefingi,
- aktywności automatyzacji.

To może być zarówno ekran dla klienta, jak i account management view dla zespołu.

8. Client onboarding i customer success

Brakuje warstwy post-sale:

- onboarding checklisty,
- milestones,
- ownership po stronie agency,
- health score,
- feedback / NPS / CSAT,
- risk flags typu „brak odpowiedzi”, „opóźniona płatność”, „niska aktywność”.

9. Lepsze preferencje komunikacji

Obecne toggles są dobre jako start, ale można dodać:

- preferowany język komunikacji,
- okna czasowe dla SMS,
- digest frequency,
- granularne zgody per event class,
- audit log zmian preferencji widoczny dla supportu.

10. Ujednolicenie token flows z portalem

Briefing i sales offer mogą dalej działać bez logowania, ale warto dodać:

- możliwość „claim this in portal”,
- podpięcie tych akcji do timeline w portalu,
- spójny branding i breadcrumbs do workspace.

6.3 Kierunek architektoniczny

11. Zdecydować, czy `Client` ma dalej oznaczać wszystko

Są dwa rozsądne kierunki:

1. zostawić `Client` jako master record, ale dodać wyraźny lifecycle i typy person,
2. rozdzielić `Prospect`, `ClientAccount` i `PortalUser` na osobne byty.

Dla Website Expert na obecnym etapie sensowniejszy wydaje się wariant 1, bo wymaga mniejszego refaktoru, ale musi dostać:

- jasny lifecycle,
- poprawne policy,
- feature gating,
- spójne scoping po `business_id`.

12. Uporządkować zależność `Client <-> Business`

Pole `business_id` już istnieje w `clients`, ale model nie jest jeszcze do końca wykorzystywany jako pełnoprawna relacja domenowa.

To warto doprowadzić do końca, bo bez tego trudniej o:

- tenant safety,
- segmentację klientów per business,
- precyzyjne raporty,
- skalowanie do agency SaaS.

---

7. Rekomendowana kolejność prac

1. Naprawić portal invite flow i URL-e.
2. Ustalić zasady membership `BusinessUser` dla klientów agencyjnych.
3. Wprowadzić feature gating i poprawne policy dla portalu.
4. Zamienić `ClientResource` w realny cockpit klienta.
5. Dopiero potem rozwijać multi-contact access, onboarding i customer success.

---

8. Konkluzja

Website Expert już dziś obsługuje klienta szerzej niż typowy prosty CRM: potrafi go pozyskać, kwalifikować, prowadzić przez ofertę i kontrakt, rozliczać, komunikować się z nim i dać mu portal do współpracy. To jest silna baza.

Największy problem nie leży w braku funkcji, tylko w tym, że klient agencyjny i klient produktu SaaS zaczynają korzystać z tego samego interfejsu i części tych samych modeli, ale nie mają jeszcze domkniętych granic

dostępu, onboardingów i scoping rules. To właśnie tam są dziś najlepsze miejsca do usprawnień.